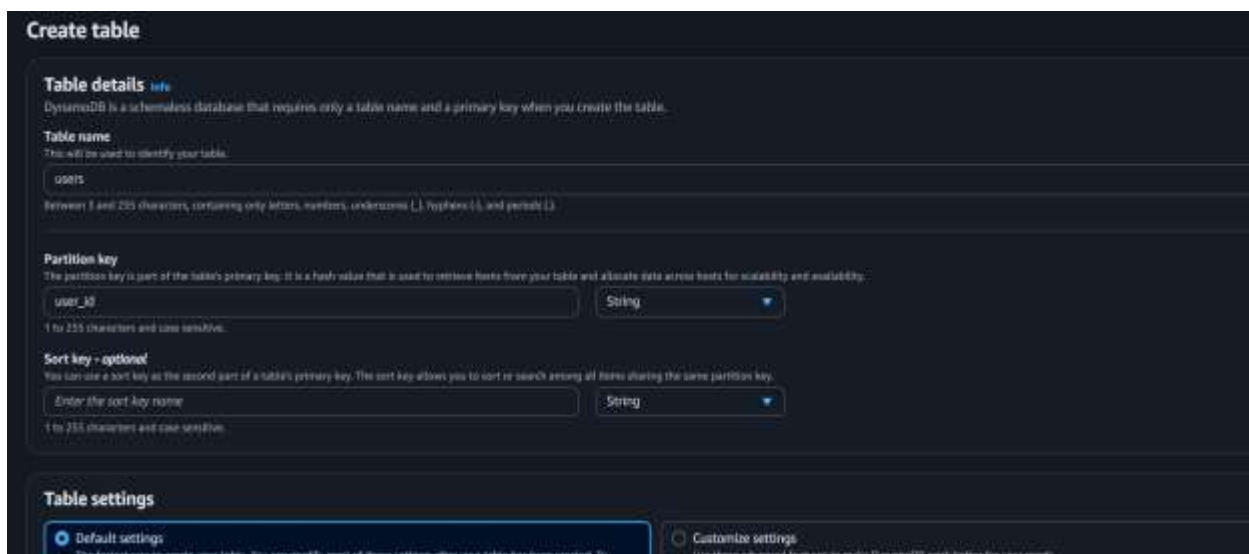


Project: Insert Data into DynamoDB using AWS Lambda

Step 1: Create DynamoDB Table

Go to AWS Console → DynamoDB → Create Table

- Table name: users
- Partition key: user_id (String)



The screenshot shows the 'Create table' wizard in the AWS Management Console. It is divided into three main sections: 'Table details', 'Partition key', and 'Table settings'. In the 'Table details' section, the 'Table name' is set to 'users'. The 'Partition key' section shows 'user_id' as the partition key with a data type of 'String'. The 'Sort key - optional' section is currently empty. At the bottom, the 'Table settings' section has 'Default settings' selected.

Create table

Table details [help](#)
DynamoDB is a schemaless database that requires only a table name and a primary key when you create the table.

Table name
This will be used to identify your table.
users
Between 3 and 255 characters, containing only letters, numbers, underscores (_), hyphens (-), and periods (.)

Partition key
The partition key is part of the table's primary key. It is a hash value that is used to retrieve items from your table and allocate data across hosts for scalability and availability.
user_id String
1 to 255 characters and case sensitive.

Sort key - optional
You can use a sort key as the second part of a table's primary key. The sort key allows you to sort or search among all items sharing the same partition key.
Enter the sort key name String
1 to 255 characters and case sensitive.

Table settings
☒ Default settings
The fastest way to create your table. You can modify most of these settings after your table has been created. To
☐ Customize settings
Use these advanced features to make DynamoDB work better for your needs.

Why this design?

- user_id = unique
- Fast lookup
- No scan needed

Step 2: Create IAM Role for Lambda

Create role with permissions:

Select trusted entity [Help](#)

Step 1: Select trusted entity
Step 2: Add permissions
Step 3: Name, review, and create

Trusted entity type

- ☒ **AWS service**
Allow AWS services like EC2, Lambda, or others to perform actions in this account.
- ☐ **AWS account**
Allow entities in other AWS accounts belonging to you or a 3rd party to perform actions in this account.
- ☐ **Web identity**
Allow users to connect by the specified external web identity provider to assume this role to perform actions in this account.
- ☐ **SAML 2.0 federation**
Allow users federated with SAML 2.0 from a supported identity to perform actions in this account.
- ☐ **Custom trust policy**
Enter a custom trust policy to enable access to perform actions in this account.

Use case
Allow an AWS service like EC2, Lambda, or others to perform actions in this account.

Service or use case
Lambda

Choose a use case for the specified service.

Use case

- ☒ **Lambda**
Allow Lambda functions to call AWS services on your behalf.
- ☐ **AWSServiceRoleForLambda**
Allow Lambda to manage AWS resources on your behalf.

Add permissions [Help](#)

Step 1: Select trusted entity
Step 2: Add permissions
Step 3: Name, review, and create

Permissions policies (1/1150) [Help](#)

Choose one or more policies to attach to your new role.

Filter by Type: All types | Applies

Policy name	Type	Description
☒ AmazonDynamoDBFullAccess	AWS managed	Provides full access to Amazon DynamoDB...
☐ AmazonDynamoDBReadOnlyAccess	AWS managed	Provides full access to Amazon DynamoDB...
☐ AmazonDynamoDBReadOnlyAccessForViz	AWS managed	This policy is on a deprecation path. See ...
☐ AmazonDynamoDBReadOnlyAccess	AWS managed	Provides read-only access to Amazon Dya...

Set permissions boundary - optional

Cancel Previous Next

Name, review, and create

Step 1: Select trusted entity
Step 2: Add permissions
Step 3: Name, review, and create

Role details

Role name
Enter a meaningful name to identify this role.
DynamoDB_Import_Role

Description
Add a brief description for this role.
Allows Lambda functions to call AWS services on your behalf.

Step 1: Select trusted entities [Help](#)

Trust policy

```

1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Effect": "Allow",
6       "Action": [
7         "sts:AssumeRole"
8       ],
9       "Principal": {
10        "Service": [
11          "lambda.amazonaws.com"
12        ]
13      }
14    }
15  ]
16 }

```

- AmazonDynamoDBFullAccess (for demo only)

In real production → use least privilege

Step 3: Create Lambda Function

- Runtime: Python 3.x
- Attach IAM role

The screenshot shows the 'Create function' page in the AWS Lambda console. At the top, there are three tabs: 'Author from scratch' (selected), 'Use a Blueprint', and 'Container image'. Below the tabs, the 'Basic information' section contains a 'Function name' field with 'insert-dynamodb' entered, a 'Runtime' dropdown set to 'Python 3.12', and a 'Permissions' section. The 'Custom settings' section at the bottom is partially visible.

The screenshot shows the 'Configure custom execution role' dialog box. It has a title bar with a close button. The 'Execution role' section shows 'DynamoDB_insert_role' selected from a dropdown. Below the dropdown are buttons for 'Create new role' and 'View role details in IAM'. At the bottom of the dialog are 'Cancel' and 'Save' buttons.

Lambda Code (Core Part)

```
import json

import boto3

import uuid

dynamodb = boto3.resource('dynamodb')

table = dynamodb.Table('users')
```

```

def lambda_handler(event, context):

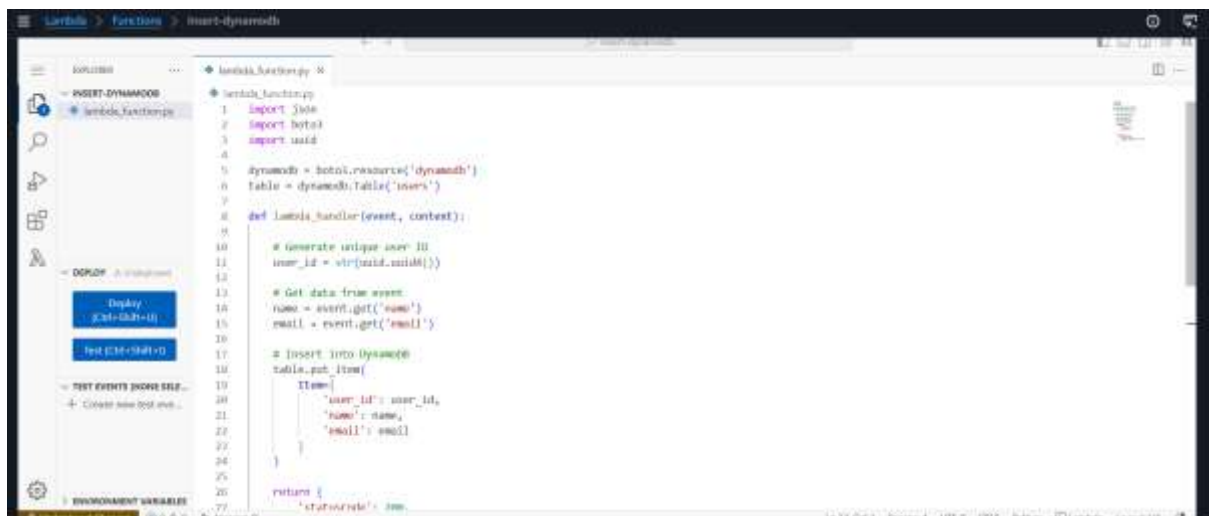
    # Generate unique user ID
    user_id = str(uuid.uuid4())

    # Get data from event
    name = event.get('name')
    email = event.get('email')

    # Insert into DynamoDB
    table.put_item(
        Item={
            'user_id': user_id,
            'name': name,
            'email': email
        }
    )

    return {
        'statusCode': 200,
        'body': json.dumps({
            'message': 'User created successfully',
            'user_id': user_id
        })
    }

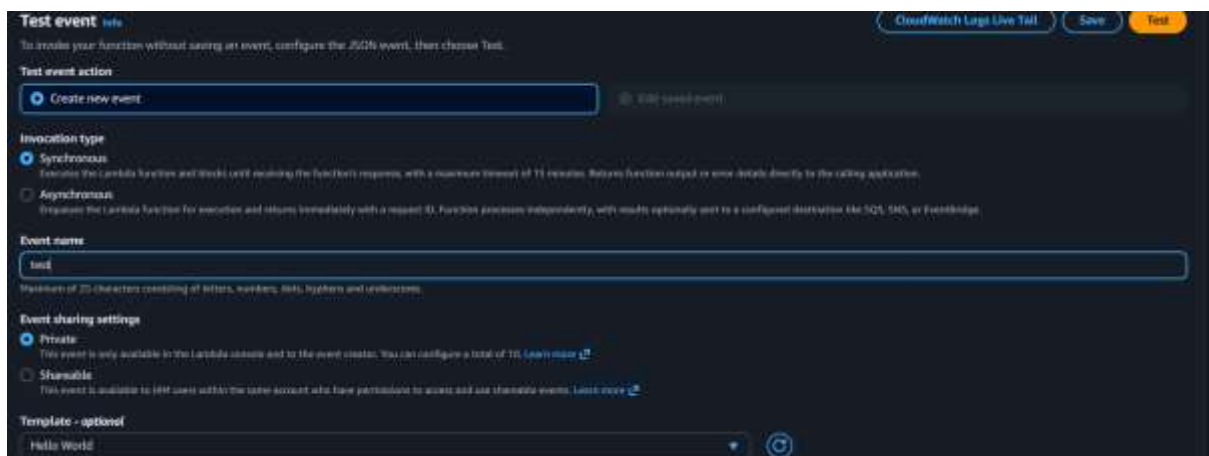
```



Step 4: Test Lambda

Use test event:

```
{  
  "name": "Shravuu",  
  "email": "shravu@example.com"  
}
```



The screenshot shows the 'Test event' configuration page in the AWS Lambda console. The 'Test event action' is set to 'Create new event'. The 'Invocation type' is set to 'Synchronous'. The 'Event name' is 'test'. The 'Event sharing settings' are set to 'Private'. The 'Template - optional' is 'Hello World'.

Test event [info](#)

To invoke your function without saving an event, configure the JSON event, then choose Test.

Test event action

☒ Create new event ☐ Use saved event

Invocation type

☒ Synchronous
Invokes the Lambda function and blocks until receiving the function response, with a maximum timeout of 15 minutes. Returns function output or error details directly to the calling application.

☐ Asynchronous
Invokes the Lambda function for execution and returns immediately with a request ID. Function processes independently, with results optionally sent to a configured destination like SQS, SNS, or EventBridge.

Event name

test

Maximum of 255 characters consisting of letters, numbers, dots, hyphens and underscores.

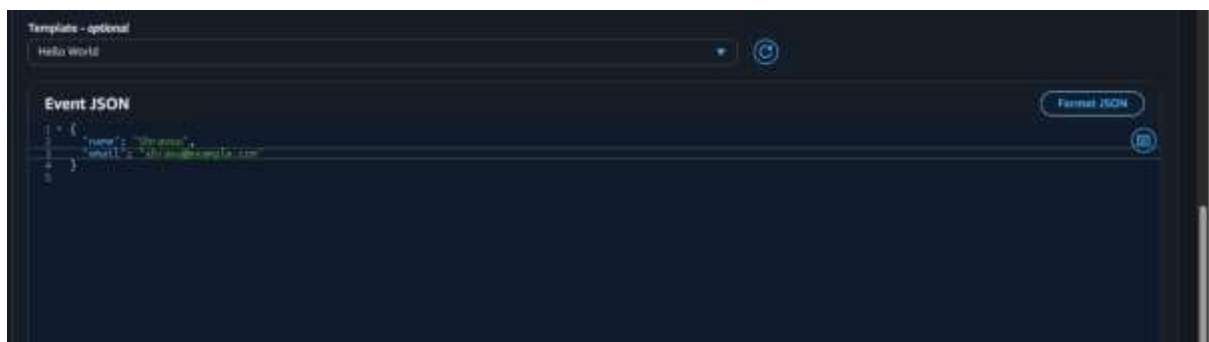
Event sharing settings

☒ Private
This event is only available to the Lambda console and to the event creator. You can configure a total of 10. [Learn more](#)

☐ Shareable
This event is available to all users within the same account who have permissions to access and use shareable events. [Learn more](#)

Template - optional

Hello World



The screenshot shows the 'Event JSON' configuration page in the AWS Lambda console. The 'Template - optional' is 'Hello World'. The 'Event JSON' is a JSON object with 'name' and 'email' fields.

Template - optional

Hello World

Event JSON

Format JSON

```
{  
  "name": "Shravuu",  
  "email": "shravu@example.com"  
}
```

Step 5: Verify in DynamoDB

- Go to table → Explore items
- You should see inserted data

[DynamoDB](#) > [Explore items: users](#) > [Edit item](#)

Edit item

You can add, remove, or edit the attributes of an item. You can nest attributes inside other attributes up to 32 levels deep. [Learn more](#)

[Form](#) [JSON view](#)

Attributes [Add new attribute](#)

Attribute name	Value	Type	Remove
id - Partition key	20119575-82e5-47e6-b056-5e71afba0c5f	String	
email	shrawu@example.com	String	Remove
name	Shrawu	String	Remove

[Cancel](#) [Save](#) [Save and close](#)

Add Second Data:

Repeat Step 4: Test Lambda:

Use test event:

```
{
  "name": "Virat Kohli",
  "email": "Virat@example.com"
}
```

Test event [Info](#) [CloudWatch Logs Live Tail](#) [Save](#) [Test](#)

To invoke your function without saving an event, configure the JSON event, then choose Test.

Test event action

[Create new event](#) [Edit saved event](#)

Invocation type

☒ **Synchronous**
Invokes the Lambda function and blocks until receiving the function response, with a maximum timeout of 10 minutes. Returns function output or error details directly to the calling application.

☐ **Asynchronous**
Invokes the Lambda function for execution and returns immediately with a request ID. Function processes independently, with results optionally sent to a configured destination like SQS, SNS, or Eventbridge.

Event name

Maximum of 20 characters consisting of letters, numbers, dots, hyphens and underscores.

Event sharing settings

☒ **Private**
This event is only available to the Lambda console and to the event creator. You can configure a total of 10. [Learn more](#)

☐ **Shareable**
This event is available to test users within the same account who have permissions to create and use shareable events. [Learn more](#)

Template - optional

[+](#) [🔄](#)

Test event [help](#) Delete CloudWatch Logs Live Tail Save Test

To invoke your function without saving an event, modify the event, then choose Test. Lambda uses the modified event to invoke your function, but does not overwrite the original event until you choose Save.

Test event action

☐ Create new event ☒ Edit saved event

Invocation type

☒ Synchronous
Invokes the Lambda function and blocks until receiving the function's response, with a maximum timeout of 15 minutes. Returns function output or error details directly to the calling application.

☐ Asynchronous
Invokes the Lambda function for execution and returns immediately with a request ID. Function processes independently, with results optionally sent to a configured destination like S3, SNS, or EventBridge.

Event name

test ↺

Event JSON Format JSON

```
{
  "name": "Virat Kohli",
  "email": "Virat@example.com"
}
```

Output:

► Filters - optional

Run Reset

Table: users - Items returned (2) ↺ Actions ▼ Create item

Scan started on May 21, 2026, 18:12:25

< 1 > ⚙

☐	user_id (String) ▼	email ▼	name ▼
☐	d3ef6135-53ee-432d...	Virat@example.com	Virat Kohli
☐	26119375-82e5-47e8...	shravu@example.com	Shravuu

Learn Here:

- Lambda invocation
- DynamoDB write operation
- IAM role usage
- Event-driven design

UPDATED LAMBDA (Dynamic Table Name + CRUD)

Change Code :

```
import json

import boto3

import uuid

dynamodb = boto3.resource('dynamodb')

# Allowed tables (IMPORTANT for security)
ALLOWED_TABLES = ['users', 'customers']

def lambda_handler(event, context):

    table_name = event.get('table_name')

    # ----- SECURITY CHECK -----

    if table_name not in ALLOWED_TABLES:

        return response("Invalid table name", {})

    table = dynamodb.Table(table_name)

    action = event.get('action')

    # ----- CREATE -----

    if action == 'create':

        user_id = str(uuid.uuid4())

        name = event.get('name')

        email = event.get('email')

        table.put_item(

            Item={

                'user_id': user_id,

                'name': name,

                'email': email

            }

        )
```



```
return response("User created", {'user_id': user_id})
```

```
# ----- UPDATE -----
```

```
elif action == 'update':
```

```
    user_id = event.get('user_id')
```

```
    name = event.get('name')
```

```
    email = event.get('email')
```

```
    table.update_item(
```

```
        Key={'user_id': user_id},
```

```
        UpdateExpression="SET #n = :name, email = :email",
```

```
        ExpressionAttributeNames={'#n': 'name'},
```

```
        ExpressionAttributeValues={
```

```
            ':name': name,
```

```
            ':email': email
```

```
        },
```

```
        ReturnValues="UPDATED_NEW"
```

```
    )
```

```
    return response("User updated", {'user_id': user_id})
```

```
# ----- DELETE SINGLE -----
```

```
elif action == 'delete':
```

```
    user_id = event.get('user_id')
```

```
    table.delete_item(
```

```
        Key={'user_id': user_id}
```

```
    )
```

```
    return response("User deleted", {'user_id': user_id})
```

```
# ----- DELETE ALL -----
```

```
elif action == 'delete_all':
```

```
    scan = table.scan()
```

with table.batch_writer() as batch:

for item in scan['Items']:

batch.delete_item(

Key={'user_id': item['user_id']}

)

return response("All records deleted", {})

else:

return response("Invalid action", {})

----- Helper -----

def response(message, data):

return {

'statusCode': 200,

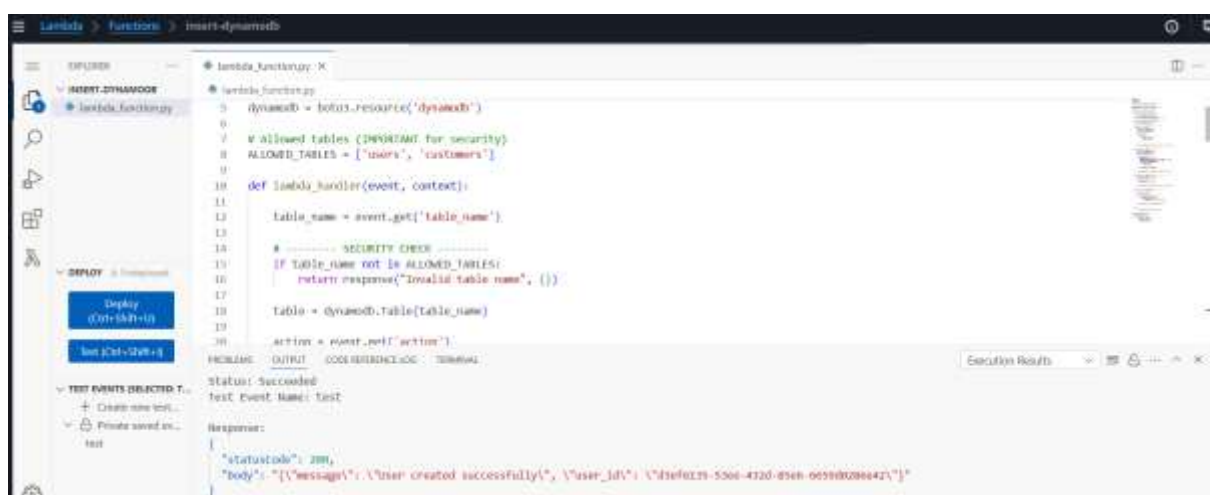
'body': json.dumps({

'message': message,

'data': data

})

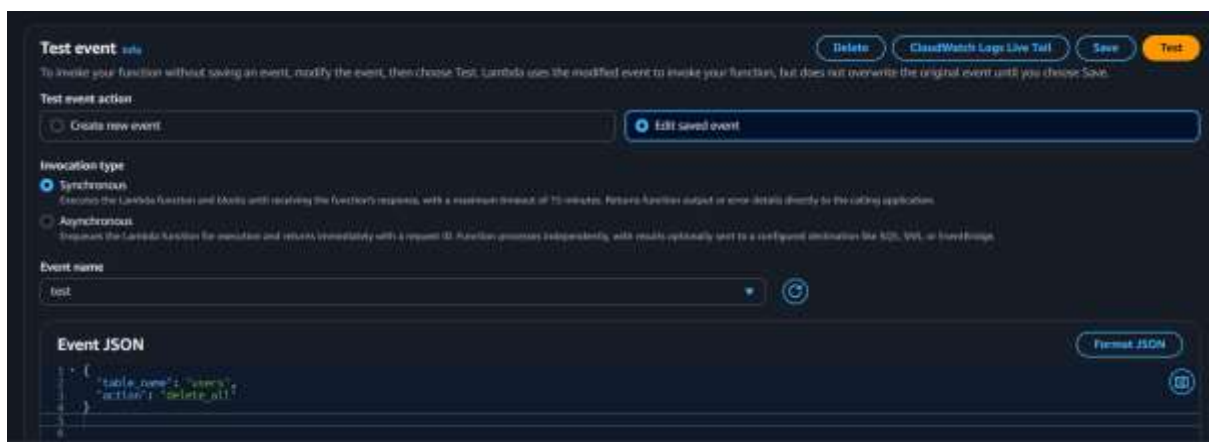
}



Test Lambda:

Delete All:

```
{  
  "table_name": "users",  
  "action": "delete_all"  
}
```



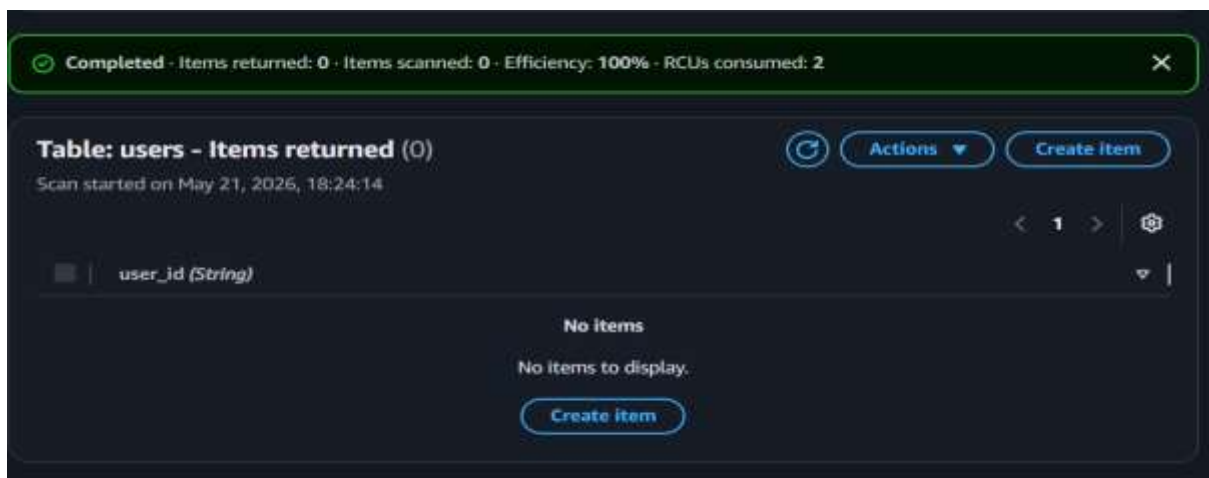
The screenshot shows the 'Test event' interface in the AWS Lambda console. At the top, there are buttons for 'Delete', 'CloudWatch Logs Live Tail', 'Save', and 'Test'. Below this, a note states: 'To invoke your function without saving an event, modify the event, then choose Test. Lambda uses the modified event to invoke your function, but does not overwrite the original event until you choose Save.' The 'Test event action' section has two radio buttons: 'Create new event' (selected) and 'Edit saved event'. The 'Invocation type' section has two radio buttons: 'Synchronous' (selected) and 'Asynchronous'. The 'Event name' field contains 'test'. The 'Event JSON' section shows a JSON object:

```
{  
  "table_name": "users",  
  "action": "delete_all"  
}
```

 with a 'Format JSON' button.

Output:

Delete All Files:



The screenshot shows the output of a Lambda function invocation. At the top, a green status bar indicates 'Completed - Items returned: 0 - Items scanned: 0 - Efficiency: 100% - RCUs consumed: 2'. Below this, the title 'Table: users - Items returned (0)' is displayed, along with 'Scan started on May 21, 2026, 18:24:14'. The table structure is shown with a single column 'user_id (String)'. The message 'No items' and 'No items to display.' is shown, with a 'Create item' button at the bottom.

Create:

```
{
  "table_name": "users",
  "action": "create",
  "name": "Shravu",
  "email": "poul@example.com"
}
```

Test event [Info](#)

Delete

CloudWatch logs Live Tail

Save

Test

To invoke your function without saving an event, modify the event, then choose Test. Lambda uses the modified event to invoke your function, but does not overwrite the original event until you choose Save.

Test event action

Create new event

Edit saved event

Invocation type

Synchronous
Execute the Lambda function and blocks until receiving the function's response, with a maximum timeout of 15 minutes, returns function output or error details directly to the calling application.

Asynchronous
Execute the Lambda function for execution and returns immediately with a request ID. Function processes independently, with results optionally sent to a configured destination like S3, SNS, or EventBridge.

Event name

test

Event JSON

Format JSON

```
{
  "table_name": "users",
  "action": "create",
  "name": "burke",
  "email": "colleen@sample.com"
}
```

Output:

Create File:

Select a table or index

Table - users

Select attribute projection

All attributes

► Filters - optional

Run

Reset

Table: users - Items returned (1)

Scan started on May 21, 2026, 18:26:29

↺

1

↻

	user_id (String)	email	name
	7fac900-3340-4dc4-a2bc-f6c0c28843c0	poul@example.c...	Shravu

